



Solana Foundation: payment-channels [9c97d575]

Security Review

Cantina Managed review by:
Shotes, Lead Security Researcher
Slowfi, Lead Security Researcher

July 29, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Informational	4
3.1.1	A valid 28 recipient channel cannot be distributed through the direct transaction flow	4
3.1.2	Legacy 248-byte channels require an emergency upgrade to recover their funds and rent	4
3.1.3	HTTP challenge can select the program receiving payer privileges	5
3.1.4	X402 payouts can be permanently redirected to treasury when a beneficiary ATA is unusable	5
3.1.5	TypeScript self-send settlement omits the required Ed25519 verification instruction	6
3.1.6	Rust CPI builder swaps signer and writable flags for remaining accounts	6
3.1.7	Open-slot-window ratchet is not enforced across upgrades	7
3.1.8	CI publishes a non production program artifact without gating the verified build	7
3.1.9	Recoverable rent does not eliminate per-request setup and sponsor capital requirements for x402 upto	8

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

The Solana Foundation is a non-profit foundation based in Zug, Switzerland, dedicated to the decentralization, adoption, and security of the Solana ecosystem.

From Jul 14th to Jul 15th the Cantina team conducted a review of [payment-channels](#) on commit hash [0c07d575](#). The team identified a total of **9** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	0	0	0
Informational	9	0	9
Total	9	0	9

2.1 Scope

The security review had the following components in scope for [payment-channels](#) on commit hash [0c07d575](#):

```
program/payment_channels/src
├── constants.rs
├── errors.rs
├── lib.rs
├── events
│   ├── mod.rs
│   ├── opened.rs
│   └── payout_redirected.rs
├── instructions
│   ├── distribute.rs
│   ├── mod.rs
│   ├── open.rs
│   ├── reclaim.rs
│   ├── seal.rs
│   ├── settle_and_seal.rs
│   ├── withdraw_payer.rs
│   └── helpers
│       ├── hash.rs
│       ├── mod.rs
│       ├── token.rs
│       ├── voucher.rs
│       ├── accounts
│       │   ├── validation.rs
│       │   └── view.rs
│       └── ed25519
│           └── parse.rs
└── state
    ├── channel.rs
    ├── common.rs
    └── mod.rs
```

3 Findings

3.1 Informational

3.1.1 A valid 28 recipient channel cannot be distributed through the direct transaction flow

Severity: Informational.

Context: [distribute.rs#L219-L227](#), [distribution.rs#L10-L11](#).

Description: The program accepts distribution plans containing up to 32 recipients. Every `distribute` invocation must repeat the complete committed distribution preimage and provide one token account for every recipient. The program hashes the complete preimage and compares it with `Channel::distribution_hash`, and it requires the remaining-account count to equal the complete recipient count. Consequently, a distribution cannot be split into smaller direct transactions.

Using the committed TypeScript client, a one-signature versioned transaction, and an address lookup table containing every eligible non-signer account, a 27-recipient `open` is 1,182 bytes and its `distribute` is 1,202 bytes. At 28 recipients, `open` is 1,216 bytes and still fits the 1,232-byte transaction limit, but `distribute` is 1,238 bytes before adding any compute-budget instruction. LiteSVM tests do not detect this because they accept a transaction object without enforcing the serialized network packet limit.

A payer can therefore open and fund a valid 28-recipient channel and advance its settled watermark, after which every supported direct `distribute` transaction is rejected before program execution. The payer can recover only the amount above `settled`; paying the settled merchant allocation requires a custom CPI adapter or a program upgrade. This can leave an arbitrary settled amount unavailable through the documented direct user flow.

Recommendation: Reduce the accepted recipient limit to a value whose `open` and `distribute` transactions fit when built by the supported client, including the required compute-budget instructions and expected signer configuration. Treat 27 as an upper bound until stricter serialization tests establish a lower production-safe limit.

Preferably redesign distribution so that payouts can be processed in authenticated chunks without repeating the complete plan. Add client-level tests that serialize the actual versioned transactions at every supported boundary and reject configurations exceeding 1,232 bytes.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.1.2 Legacy 248-byte channels require an emergency upgrade to recover their funds and rent

Severity: Informational.

Context: [channel.rs#L150-L159](#), [channel.rs#L321-L340](#), [transmutable.rs#L45-L55](#).

Description: The current `Channel` structure appends an eight-byte `open_slot` field and has a length of 256 bytes. The stored channel version remains `1`, and no compatibility loader or migration path was added for the previous 248-byte version 1 layout.

Every channel entry point calls the current zero copy loader before validating the discriminator or version. The loader requires the account data length to equal the current type length exactly, so a 248-byte account returns `InvalidAccountData` before any instruction specific recovery logic can run.

Mainnet currently contains four program-owned 248-byte channels. All four decode as open version-1 USDC channels with `deposit = 10,000` and `settled = 0`; their initialized escrows each hold 10,000 base units. The current program rejects these accounts, leaving a total of 0.04 USDC plus channel and escrow rent unavailable through ordinary user instructions. The program remains upgradeable, which limits the severity because the operator can deploy a compatibility recovery path.

Recommendation: Add a length-aware legacy loader and a dedicated migration or close path for the 248-byte layout. The legacy path must reconstruct the previous PDA seeds rather than interpreting legacy bytes as the current structure.

Bump the stored schema version for every persisted layout change and add upgrade tests that begin with serialized accounts produced by the previously deployed release.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.1.3 HTTP challenge can select the program receiving payer privileges

Severity: Informational.

Context: [002-http-protocol.md#L62-L65](#), [002-http-protocol.md#L110-L111](#).

Description: The HTTP protocol makes `methodDetails.channelProgram` a required challenge field. Its transaction-validation instructions then require only that the submitted `open` instruction target the program named by that same challenge. The expected program is therefore selected by wire data supplied by the resource server instead of being derived from a trusted network-to-program mapping in the client.

An `open` instruction forwards payer signer privileges and writable token accounts to its target program. A malicious server can advertise its own program address and cause a client that follows the document literally to construct and sign an `open`-shaped transaction for that program. Matching the instruction target to the untrusted challenge value does not authenticate the target.

The attack requires a client to trust a malicious server's challenge and to omit an independent canonical-program check. If those conditions hold, the attacker program receives the payer signature and writable accounts and can attempt token-program CPIs outside the intended channel setup. This creates a possible wallet asset-loss path, but it is confined to clients implementing the draft HTTP guidance without additional transaction policy.

Recommendation: Remove `channelProgram` from untrusted challenge negotiation. Define one canonical payment-channel program for each supported network in trusted client configuration and require clients, servers, and sponsors to reject every transaction targeting another address.

Add client tests in which a challenge advertises a different program while all other `open` fields are valid, and verify rejection before requesting the payer signature.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.1.4 X402 payouts can be permanently redirected to treasury when a beneficiary ATA is unusable

Severity: Informational.

Context: [view.rs#L355-L417](#).

Description: For a nonzero payout, `payout_destination` redirects the transfer to the treasury when the beneficiary's canonical token account is closed, malformed, uninitialized, frozen, reassigned, or contains an unsupported Token-2022 extension. After transfers succeed, an open-channel distribution advances `payout_watermark` to the current settled amount. Repairing the beneficiary account later does not restore the redirected delta.

This is an explicit forfeiture policy for the generic payment-channel protocol and is not an authorization bypass: the cranker cannot substitute a noncanonical account, and the redirected amount is observable through an event. The concern arises for x402 integrations only if their `payTo` security property requires settled funds to reach that beneficiary without redirection.

Under that x402 assumption, a permissionless distribution executed while the canonical `payTo` ATA is unusable can irreversibly deliver the merchant's settled share to treasury. The trigger depends on beneficiary-account state and on the x402 no-redirection requirement being normative, so the issue is rated Low conservatively.

Recommendation: Choose one payout policy explicitly for each integration. If x402 requires no redirection, make a nonzero unusable `payTo` destination revert without advancing its payout watermark, or provide a retryable per-beneficiary claim mechanism. If forfeiture remains intentional, update the x402 requirements and require beneficiary ATA health checks before vouchers are accepted and before distribution is cranked.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.1.5 TypeScript self-send settlement omits the required Ed25519 verification instruction

Severity: Informational.

Context: [paymentChannels.ts#L325-L336](#), [voucher.rs#L22-L43](#).

Description: The generated TypeScript plugin adds transaction planning and sending helpers directly to `settle` and `settleAndSeal`. These helpers receive only the payment-channel instruction inputs and therefore construct a transaction plan without voucher signature material or an Ed25519 precompile instruction.

On-chain voucher verification loads the immediately preceding top-level instruction and requires it to be the Ed25519 program with the canonical voucher layout. `settle` has no voucher-free path, while `settleAndSeal` performs this verification whenever `hasVoucher` is nonzero.

Consequently, every standalone fluent `settle(...).sendTransaction()` call fails with `MissingEd25519Verification`, and voucher-bearing fluent `settleAndSeal` calls fail in the same way. This is fail-closed and does not lose funds because callers can manually compose the precompile and settlement instructions, but the generated high-level settlement interface is unusable as exposed.

Recommendation: Provide a settlement transaction-plan helper that accepts the voucher, public key, and signature and prepends the canonical Ed25519 instruction. Until that exists, remove self-send helpers from `settle` and from voucher-bearing `settleAndSeal`, and document the required manual composition beside the generated API.

Add an end-to-end TypeScript test that sends a voucher-bearing settlement through the supported high-level interface.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.1.6 Rust CPI builder swaps signer and writable flags for remaining accounts

Severity: Informational.

Context: [distribute.rs#L433-L438](#), [distribute.rs#L598-L608](#).

Description: The generated Rust CPI builder documents and stores every remaining account as `(account, is_writable, is_signer)`. The invocation path consumes the same tuple in the opposite order, assigning tuple element one to `is_signer` and element two to `is_writable`.

`distribute` is the current instruction whose remaining accounts are semantically required: they are the recipient token accounts. A caller following the documented builder API and adding a writable non-signer recipient account produces an inner meta that is signer and readonly. The CPI normally fails its privilege check before the payment-channel program can process the distribution.

This makes the generated Rust CPI distribution path fail closed for correctly supplied recipient accounts. Top-level Rust and TypeScript instruction builders are unaffected, and callers can work around the issue by constructing the instruction metas directly, so the impact is limited to CPI integrations using the generated builder.

Recommendation: Fix the Rust renderer so the stored and consumed tuple conventions agree, then regenerate all Rust instruction clients. Add a renderer-level test asserting that `(is_writable = true, is_signer = false)` produces a writable non-signer `AccountMeta`.

Do not patch only the generated `distribute.rs`, because regeneration would restore the defect and the same template is emitted for other CPI builders.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.1.7 Open-slot-window ratchet is not enforced across upgrades

Severity: Informational.

Context: `constants.rs#L39-L57`.

Description: Full channel deallocation relies on `OPEN_SLOT_WINDOW` never increasing across program versions. A channel address can be deallocated only after its original `open_slot` is outside the active window; keeping every later window no larger prevents the same address from becoming valid again and prevents old address-bound vouchers from applying to a later incarnation.

The current guard compares `OPEN_SLOT_WINDOW` with `MAX_DEPLOYED_OPEN_SLOT_WINDOW`, but both are editable constants in the same source revision. Raising both values in one upgrade satisfies the compile-time assertion and does not consult any immutable historical value.

An accidental future increase can make an already deallocated seed tuple valid again during the widened interval. Reopening that address with the same parties and signer can make a previously signed voucher relevant to newly deposited funds. This requires an unsafe program upgrade and a matching reincarnation, so it is an upgrade-process risk rather than a current unprivileged exploit.

Failure example scenario:

1. Version A uses a window of 100 slots.
2. A channel opens at slot 1,000.
3. After slot 1,100, the channel closes and its PDA is deallocated.
4. Version B increases the window to 200 slots.
5. At slot 1,150, `open_slot` = 1,000 becomes valid again because $1,150 - 1,000 \leq 200$.
6. The payer reuses the same payer, payee, mint, voucher signer, salt and `open_slot`.
7. These inputs recreate the exact same channel PDA.
8. An old voucher only sees that channel address. If it has not expired and fits the new deposit, it can settle against the newly funded channel.

Recommendation: Enforce the historical ceiling in release automation by comparing the proposed value with the value in the previously deployed source or artifact. For stronger protection, persist a non-increasing ceiling in program state and have `open` reject configurations that exceed it.

Add an upgrade regression test that closes an account under the previous window, attempts to increase the window, and proves that the old `open_slot` cannot become admissible again.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.1.8 CI publishes a non production program artifact without gating the verified build

Severity: Informational.

Context: `ci.yml#L31-L49`, `justfile#L148-L162`.

Description: The main CI workflow runs `just build-program` and uploads `target/deploy/payment_channels.so` under the generic artifact name `program-so`. That build uses the default localnet feature configuration. The production path is a separate `verified-build` recipe that selects the mainnet feature and a pinned verifiable-build image, but CI does not run that recipe or bind its result to the uploaded artifact.

The localnet binary contains configuration that is intentionally different from the mainnet build, including the sentinel treasury owner. The uploaded artifact name and workflow do not prevent an operator or downstream job from treating it as the reviewed deployable program.

If the generic CI artifact is used in a production deployment, treasury-routed token amounts can be sent according to the localnet configuration rather than the intended mainnet owner. This requires an operational artifact-selection mistake rather than an on-chain attacker, so the issue is rated Low.

Recommendation: Add a blocking CI job that produces the verified mainnet-feature binary with the pinned build image and locked dependencies. Publish it under an explicit production artifact name together with its executable hash, source commit, feature set, and build-image digest.

Keep localnet artifacts clearly separated and prevent deployment recipes from consuming a shared unqualified `target/deploy/payment_channels.so` path.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.1.9 Recoverable rent does not eliminate per-request setup and sponsor capital requirements for x402 upto

Severity: Informational.

Context: [001-payment-channel-state-machine.md#L250-L258](#), [open.rs#L334-L371](#), [reclaim.rs#L40-L54](#), [README.md#L33-L40](#).

Description: The epoch-addressed channel design fixes the previous permanent rent cost: after a channel is distributed and the open-slot window has elapsed, `reclaim` fully deallocates the channel PDA and returns its lamports to the recorded rent payer. This removes the 897,840-lamport tombstone that the earlier design left behind.

It does not, however, remove the fresh on-chain setup from the one-shot x402 upto flow. Each request still creates a new 256-byte channel PDA and escrow associated token account, and the protected resource is served only after the `open` transaction has been broadcast and the channel is confirmed `Open`. After the request, the facilitator must settle, seal, and distribute the channel. Distribution closes the escrow token account, but the channel PDA remains allocated until `clock.slot > open_slot + OPEN_SLOT_WINDOW` unless the client backdates `open_slot` enough to make immediate deallocation possible. Backdating reduces the rent-retention interval at the cost of narrowing the setup transaction's landing window.

Using the current mainnet rent-exempt minimums, the sponsor initially fronts 2,672,640 lamports for the 256-byte channel and 2,039,280 lamports for its 165-byte escrow token account, or 4,711,920 lamports per request before transaction fees and the token deposit. The previous 248-byte channel required 4,656,240 lamports upfront, so the new design reduces the eventual net rent cost but increases the initial rent requirement by 55,680 lamports (approximately 1.2%). At 100 new channels per second and a full approximately 600-second PDA-retention interval, the channel-PDA component alone requires approximately 160.3584 SOL of circulating sponsor capital. If escrow token accounts also remain open for that interval, total rent float reaches approximately 282.7152 SOL. In practice the escrow component depends on request and settlement latency, while delayed reclaim sweeps can make the PDA component higher. Because reclaim returns rent only to the recorded rent payer and pays no caller reward, an operator must also index distributed channels and reliably run the cleanup process.

No rent is permanently lost when cleanup runs successfully, and this is not an attacker-driven loss of funds. Nevertheless, the one-shot x402 upto profile still incurs an on-chain confirmation before every protected request, requires fresh sponsor liquidity for every concurrent channel, and creates an ongoing indexing and reclamation obligation. For low-value or high-throughput requests these requirements can dominate the payment amount and limit operational scale; the reusable session or batch-settlement model is the path that actually amortizes channel creation across multiple requests.

Recommendation: Document the difference between eventual net rent cost and peak sponsor working capital for each supported payment profile. Publish a capacity model such as $\text{request_rate} \times (\text{PDA_rent} \times \text{reclaim_delay} + \text{ATA_rent} \times \text{open_lifetime})$, together with transaction fees and token-deposit requirements, and benchmark the setup-confirmation latency expected before a resource can be served.

For high-volume or low-value workloads, prefer a reusable session or batch-settlement channel that accepts many off-chain cumulative vouchers per setup. If one-shot `upto` remains supported, provide an operator service that indexes distributed channels, batches `reclaim` calls, and alerts on unreclaimed rent. Treat backdating and any reduction of `OPEN_SLOT_WINDOW` as an explicit availability-versus-capital trade-off rather than relying on them as the primary scaling mechanism.

Solana Foundation: Acknowledged.

Cantina Managed: Acknowledged.